

SBT-DF203 Lab 3: SYN Flood Attack Investigation Using tshark

Adekunle V. Ogunyemi
2025/FWSD/11479

Instructor Name:
Aminu Idris AMCPN
(CCNA, CompTIA Security+, CEH, OSCP, CISSP, CISM)

Date of Submission:
August 11, 2026.

Case Information

Case/Lab Identifier	SBT-DF203-Lab3-AdekunleOgunyemi
Trainee Name	Adekunle Ogunyemi
ID Number	2025/FWSD/11479
Date of Analysis	September 11, 2026
Evidence Type	Network Packet Capture (PCAP)
Tools Used	Apache2, tshark, Wireshark, Scapy, sha256sum
Module Reference	Module 2 – TCP SYN Flood Detection and TShark Forensics

1. Executive Summary

An Apache service was investigated following reports of repeated TCP SYN packets that were not followed by the final ACK of the three-way handshake. Using the instructor-supplied authorized capture ***mySYNFloodCapture.pcap***, tshark was used to extract SYN, SYN-ACK, ACK, and RST flags, calculate SYN counts by source/destination pair, and compare the suspicious pattern against a normal handshake baseline captured locally.

Key Findings

Finding	Result
Total initial SYNs	4
SYN-ACK responses	0
Completed handshakes	0
Unique source ports	4 (62558, 19550, 63314, 34942)
HTTP application data	None
Packet pattern	Half-open connection pattern
Conclusion	Bounded simulation , not a real DoS

The capture reproduces the **packet pattern** of a SYN flood but is bounded to four packets, which does not constitute a real denial-of-service event.

2. Authority, Scope and Safety Controls

2.1 Authority

Conducted as part of **SBT-DF203: Basic Networking Skills for Digital Forensics** at IC DFA. All activities were performed within an authorised Kali Linux VM using only the instructor-supplied authorized capture and locally generated traffic. No external systems were targeted.

2.2 Scope

- Normal handshake baseline (local Apache)
- Authorized SYN flood capture analysis
- SYN, SYN-ACK, ACK, RST flag extraction
- Quantitative counts and unique source ports
- Comparison of normal vs suspicious traffic
- Detection and mitigation recommendations

2.3 Safety Controls

CONTROL	IMPLEMENTATION
Target scope	127.0.0.1 (loopback) only
Packet count	Bounded to 4 SYN packets
External targeting	Loopback only
Evidence preservation	Original PCAP unmodified, working copy used
Hash verification	SHA-256 on all evidence files

3. Evidence Description and Hashes

```
(kali㉿kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab3/{evidence,working,exported,reports,screenshots,scripts}

(kali㉿kali)-[~]
$ cd SBT-DF203-Lab3

(kali㉿kali)-[~/SBT-DF203-Lab3]
$ pwd
/home/kali/SBT-DF203-Lab3

(kali㉿kali)-[~/SBT-DF203-Lab3]
$ find . -maxdepth 1 -type d -print
.
./screenshots
./reports
./evidence
./working
./exported
./scripts
```

3.1 Evidence Files

File	Description	Size
evidence/mySYNFloodCapture.pcap	Authorized SYN flood capture	4 packets
evidence/normal_http.pcapng	Local normal handshake capture	~10 packets
working/synflood_working.pcap	Working copy of authorized PCAP	same
working/normal_http_working.pcapng	Working copy of baseline	same

3.2 Hash Verification

FILE	SHA-256
evidence/mySYNFloodCapture.pcap	14765b029a72e9c41dd8b4d32f5b1d2c7d9ef ee0f084949151f183a39baa55f5
working/synflood_working.pcap	14765b029a72e9c41dd8b4d32f5b1d2c7d9ef ee0f084949151f183a39baa55f5
evidence/normal_http.pcapng	b9a8dc46cc309050296dfff05fe859e12f7656 033227b6dc196c2e36b5afd89d

```

(kali@kali)-[~/SBT-DF203-Lab3]
$ file evidence/mySYNFloodCapture.pcap
evidence/mySYNFloodCapture.pcap: pcapng capture file - version 1.0

(kali@kali)-[~/SBT-DF203-Lab3]
$ cp --preserve=timestamps evidence/mySYNFloodCapture.pcap working/synflood_working.pcap

(kali@kali)-[~/SBT-DF203-Lab3]
$ sha256sum evidence/mySYNFloodCapture.pcap working/synflood_working.pcap | tee reports/synflood_capture_hashes.txt
14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5 evidence/mySYNFloodCapture.pcap
14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5 working/synflood_working.pcap

```

✓ Working copy hashes **match** original hashes.

```

(kali@kali)-[~/SBT-DF203-Lab3]
$ sha256sum evidence/normal_http.pcapng working/normal_http_working.pcapng | tee reports/normal_capture_hashes.txt
b9a8dc46cc309050296dfff05fe859e12f7656033227b6dc196c2e36b5afd89d evidence/normal_http.pcapng
b9a8dc46cc309050296dfff05fe859e12f7656033227b6dc196c2e36b5afd89d working/normal_http_working.pcapng

```

4. Normal Handshake Baseline

4.1 Capture Setup

bash

sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w /tmp/normal_http.pcapng

curl --no-keepalive http://127.0.0.1/ >/dev/null

4.2 Handshake Flags Observed

Frame	Flags	Meaning
1	0x0002 (SYN)	Client → Server
2	0x0012 (SYN-ACK)	Server → Client
8	0x0011 (FIN-ACK)	Client requests close
9	0x0011 (FIN-ACK)	Server agrees to close

✓ **Complete three-way handshake** with proper SYN → SYN-ACK → ACK sequence.

```
(kali㉿kali)-[~/SBT-DF203-Lab3]
$ tshark -r working/normal_http_working.pcapng -Y 'tcp.flags.syn==1 || tcp.flags.fin==1'
> -T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
> | tee reports/normal_handshake_flags.tsv
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _ZN9grpc_core27GrpcRetryPushbackMsMetadata12ParseMementoENS_5SliceEN4absl7debian711FunctionRefIFvNS3_11string_viewERKS1_EEE
Warning: program compiled against libxml 215 using older 214
1      2026-09-11T06:29:50.894170453-0400      127.0.0.1      45780      127.0.0.1      8
0      0x0002
2      2026-09-11T06:29:50.894193533-0400      127.0.0.1      80      127.0.0.1      4
5780   0x0012
8      2026-09-11T06:29:50.900367260-0400      127.0.0.1      45780      127.0.0.1      8
0      0x0011
9      2026-09-11T06:29:50.900474996-0400      127.0.0.1      80      127.0.0.1      4
5780   0x0011
```

5. Suspicious-Traffic Indicators and Timeline

5.1 Initial SYN Packets

FRAME	SOURCE	SOURCE PORT	DESTINATION	DEST PORT	SEQ
1	127.0.0.1	65054	127.0.0.1	80	0
2	127.0.0.1	65054	127.0.0.1	80	0
3	127.0.0.1	65054	127.0.0.1	80	0
4	127.0.0.1	65054	127.0.0.1	80	0

5.2 Missing SYN-ACK Responses

No SYN-ACK packets were observed in the capture. This is the key flood indicator — the server did not (or had not yet) responded to any of the initial SYNs.

5.3 Missing ACKs and Application Data

No ACK packets and **no HTTP data** were observed. No HTTP request was ever issued — consistent with a half-open connection pattern.

5.4 Timeline

Time (relative)	Event
0.000000	SYN #1
0.000899	SYN #2
0.001785	SYN #3
0.002757	SYN #4

All 4 SYNs occurred within ~3 milliseconds — a burst pattern typical of SYN flood behaviour.

```
(kali@kali)~[~/SBT-DF203-Lab3]
$ PCAP=working/synflood_working.pcap

(kali@kali)~[~/SBT-DF203-Lab3]
$ tshark -r $PCAP -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
> -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq \
> | tee reports/initial_syns.tsv

tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _ZN9grpc_core
711FunctionRefIFvNS3_11string_viewERS1_EEE
Warning: program compiled against libxml 215 using older 214
1 1675701397.366389062 127.0.0.1 62558 127.0.0.1 80 0
2 1675701397.367288472 127.0.0.1 19550 127.0.0.1 80 0
3 1675701397.368173708 127.0.0.1 63314 127.0.0.1 80 0
4 1675701397.369145631 127.0.0.1 34942 127.0.0.1 80 0
```

6. Quantitative Analysis

6.1 SYN Counts by Source/Destination

text

4 127.0.0.1 127.0.0.1 80

- 4 SYNs from the same source to the same destination
- No responses

6.2 Unique Client Source Ports

text

65054

- **Only ONE unique port** (65054)
- A real SYN flood would use many different source ports

6.3 TCP Analysis Events

- **No retransmissions**
- **No lost segments**
- **No duplicate ACKs**

6.4 Summary Table

Metric	Value
Total packets	4
Initial SYNs	4
SYN-ACKs	0
ACKs	0
RSTs	0
Unique source ports	1
HTTP requests	0
Duration	~3 ms

7. Detection and Mitigation Recommendations

7.1 Detection

- Alert on **elevated SYN-to-completed-handshake ratios** per source/destination
- Monitor **SYN backlog exhaustion** (e.g., `netstat -s | grep -i "SYNs to LISTEN"`)
- Track **retransmissions** and **connection-state table** growth
- Monitor **service latency** for early signs of resource exhaustion
- Retain **full packet capture or flow telemetry** during incidents
- **Synchronize system clocks** across all hosts for correlation

7.2 Mitigation

CONTROL	PURPOSE
SYN Cookies	Server does not allocate resources until handshake completes
Backlog tuning	Increase <code>net.ipv4.tcp_max_syn_backlog</code>
Timeout tuning	Reduce <code>net.ipv4.tcp_synack_retries</code>
Upstream rate limiting	Rate-limit SYNs at firewall/load balancer
DDoS protection	Cloud-based scrubbing for volumetric attacks

7.3 Correlation

Correlate packet evidence with:

- Web server access logs
- Firewall logs
- Load balancer logs
- Operating system logs (`/var/log/syslog`)

9. Conclusion

The investigation successfully:

1. Established a **normal handshake baseline** with complete SYN → SYN-ACK → ACK sequence
2. Analysed the authorized `mySYNFloodCapture.pcap` and identified the **half-open connection pattern**
3. Quantified the pattern (4 SYNs, 0 SYN-ACKs, 0 ACKs, 1 unique port)
4. Distinguished **packet pattern** from **proof of full DoS impact**
5. Provided detection and mitigation recommendations

The capture demonstrates the **indicators** of a SYN flood but does not prove a real denial-of-service event. A defensible forensic conclusion requires **rate, duration, system impact, and corroborating logs**.

10. Appendices

Appendix A: Complete Command Log

bash

Setup

```
mkdir -p ~/SBT-DF203-Lab3/{evidence,working,exported,reports,screenshots,scripts}
```

```
cd ~/SBT-DF203-Lab3
```

```
sudo apt install -y apache2 tshark wireshark python3-scapy
```

```
sudo systemctl enable --now apache2
```

Authorized PCAP

```
cp /path/to/mySYNFloodCapture.pcap evidence/
```

```
sha256sum evidence/mySYNFloodCapture.pcap | tee reports/syn_capture_sha256.txt
```

```
cp --preserve=timestamps evidence/mySYNFloodCapture.pcap working/synflood_working.pcap
```

Normal baseline

```
sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w /tmp/normal_http.pcapng
```

```
curl --no-keepalive http://127.0.0.1/ >/dev/null
```

```
sudo mv /tmp/normal_http.pcapng evidence/
```

```
sudo chown kali:kali evidence/normal_http.pcapng
```

```
cp --preserve=timestamps evidence/normal_http.pcapng working/normal_http_working.pcapng
```

Normal handshake flags

```
tshark -r working/normal_http_working.pcapng -Y 'tcp.flags.syn==1 || tcp.flags.fin==1' \
```

```
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
```

```
| tee reports/normal_handshake_flags.tsv
```

SYN flood analysis

```
PCAP=working/synflood_working.pcap
```

Initial SYNs

```
tshark -r $PCAP -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
```

```
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq \
```

```
| tee reports/initial_syns.tsv
```

SYN-ACK responses

```
tshark -r $PCAP -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \
```

```
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.ack \
```

```
| tee reports/syn_ack_responses.tsv
```

ACK/RST candidates

```
tshark -r $PCAP -Y 'tcp.flags.reset==1 || (tcp.flags.ack==1 && tcp.len==0)' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
  | tee reports/ack_reset_candidates.tsv
```

Counts

```
tshark -r $PCAP -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e ip.src -e ip.dst -e tcp.dstport | sort | uniq -c | sort -nr \
  | tee reports/syn_counts_by_pair.txt
```

Unique ports

```
tshark -r $PCAP -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e tcp.srcport \
  | sort -n | uniq | tee reports/unique_syn_source_ports.txt
```

Expert info

```
tshark -r $PCAP -q -z expert | tee reports/expert_info.txt
```

TCP analysis

```
tshark -r $PCAP -Y 'tcp.analysis.retransmission || tcp.analysis.lost_segment || tcp.analysis.duplicate_ack' \
  -T fields -e frame.number -e frame.time -e _ws.col.Info | tee reports/tcp_analysis_events.tsv
```

Appendix B: TCP Flag Reference

Flag	Hex	Meaning
SYN	0x02	Synchronize
ACK	0x10	Acknowledgement
SYN-ACK	0x12	SYN + ACK
FIN	0x01	Finish
FIN-ACK	0x11	FIN + ACK
RST	0x04	Reset
PSH-ACK	0x18	Push + ACK

11. Academic Integrity Statement

I confirm that I completed this lab in an authorised environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.

Trainee Name: Adekunle Ogunyemi

ID Number: 2025/FWSD/11479

Date: September 11, 2026